Data Structures
(CSC-211)
ASSIGNMENT NO. 1

INSTRUCTOR: Ms. NAJLA RAZA

STUDENT NAME: ABDUL SABOOR HASAN

REGISTRATION NUMBER: SP25-BCT-001

DEPARTMENT : CYBER SECURITY

COMSATS UNIVERSITY ISLAMABAD(CUI)

Data Structures.

Theory Assignment 1

Abdul Saboor Hasan SP25-BCT-001

Singly Linked Lists Tasks

1. Task 1:-

Algorithm:

We will use the recursions to form a call stack to display the log in reverse order as recursions follow fast in first out (LIFO) method.

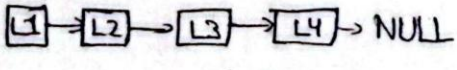
⇒ We will define a function displayReverse(Node).
If the current node is null stop the recursion and return
or else call displayReverse(node → next) to move to end of list.
- After recusive call returns, print the current node's data.
- Eend.

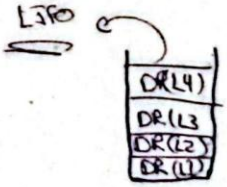
Pseudocode:- DR

```
void DisplayReverse(Node){
    If (Node == NULL){
        Return;
    }
    else{
        DisplayReverse(Node -> next) }
    print Node;
    return;
}
```

Diagram:-

if our list is like:-





DisplayReverse (L1)
DR(L2)
DR(L3)
DR(L4)

L4 L3 L2 L1

2. Task 2:-

Algorithm:-

We will write two methods:-

1. To ~~display~~ play the songs singly linked list.
2. A method to reverse the list while maintaining the addresses of songs (nodes).

1. We will use head as a starting point then traverse the list by each node the moving onto its next.
2. We will reverse the songs playlist by using the three pointer approachic.
   prev-temp - track previous node
   curr-temp - track current and processing node.
   next-temp - track next node.

- We will initialize pointers
- Repeat ~~prev = NULL~~ while curr-temp ≠ NULL{
  1. Store next node
  2. Reverse the link.
  3. Move prev-temp to one forward.
  4. Move current temp one step forward
- After loop ends ~~prev~~ head = prev.
  return.

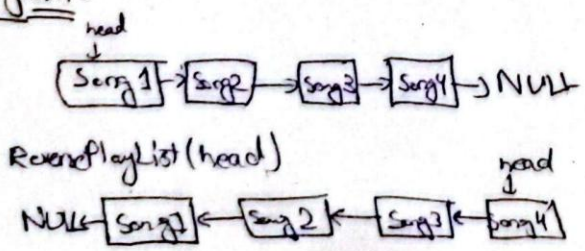
Pseudocode:-

```
void reversePlayList(head)){
    prev = NULL
    current = head
    next = NULL
    while (current ≠ NULL){
        next = current.next.
        current.next = prev
        prev = current
        current = next
    }

    head = prev;

    return;
}
```

Diagram:-



3. Task 3:

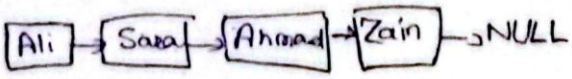
Algorithm:

- One pointer selects a contact and Another checks the rest of list for duplicates and if a duplicate is found we remove it.
- Start
- Set current pointer to the head of list.
- While current ≠ NULL:
  - set runner pointer to current.
  - While runner->next ≠ NULL.
    - If runner->next->data == current->data:
      - Store duplicate node in temp
      - Update runner->next to skip duplicate node.
      - Delete temp.
    - else move runner to next node.
  - 3. Move current to the next node.
  - 4. Stop when current become NULL.

Pseudocode:

```
void removeDuplicate(head)
    while (current ≠ NULL){
        runner = current.
        while (runner.next ≠ NULL){
            If (runner.next.data == current.data) {
                temp = runner.next
                runner.next = runner.next.next.
                Delete temp.
            else {
                rest = runner.next
                }
            }
            current = current.next.
        }
    return;
}
```

Diagram:



1. Task 4:-

Algorithm:

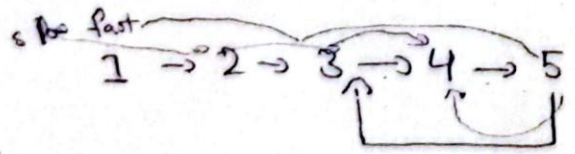
In this algorithm we use the slow fast approach.
We take two temporary pointers, slow and fast, we increment slow by one hop and fast by two hops in any case if the fast becomes equal to slow then a loop exists.

1. Start
2. Initialize two pointers, slow and fast.
3. Repeat while fast ≠ NULL OR fast.next ≠ NULL.
4. move slow one step forward and fast two steps
5. If slow == fast return true.
6. else false
7. end.

Pseudocode:-

```
void detectLoop(head){
    slow = head
    fast = head.
    while (fast ≠ NULL AND fast.next ≠ NULL){
        slow = slow.next
        fast = fast.next.next.
        if (slow == fast){
            return true;
        }
    }
    return false.
}
```

Diagram:-



true!

5. Task 5:-

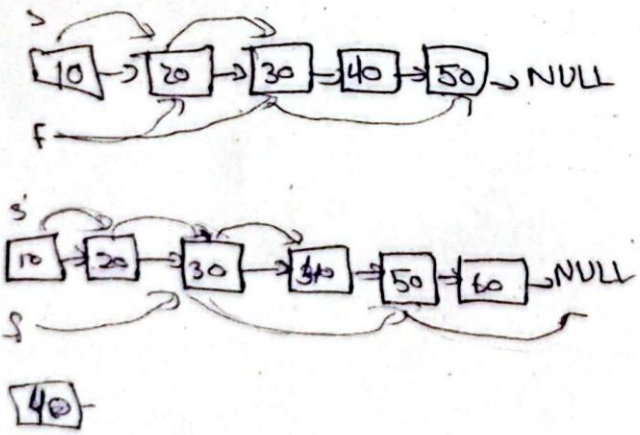
Algorithm:

In this algorithm we use the slow, fast approach, We take two temporary pointers, slow and fast, we increment slow by one hop and fast by two hops. when the fast pointer has reached NULL/end then the slow pointer will tell the mid of list.

Pseudocode:-

```
void FindMiddle(head){
        if (head == NULL){
                return NULL;
        }
        slow = head;
        fast = head;
        while (fast.next ≠ NULL AND fast.next.next ≠ NULL){
                slow = slow.next
                fast = fast.next.next.
        }
        ~~fast = head~~ fast = head;
        ~~while (fast.next ≠ slow) {~~
            ~~fast = fast.next.~~
        ~~}~~
        return ~~fast~~ slow;
}
```

Diagram:-



6. Task 6:-

Algorithm:

In this we will take two pointers and make them heads of lists. If the data in existing node is even we will add that node to even list else if it is odd then we will add it in the odd list!

Note: We are arranging existing nodes into two lists we are not creating new nodes.

Pseudocode:-

```
method sepEvenOdd(){
    odd head = NULL.
    even head = NULL.
    temp = head;
    while (temp != NULL){
        if (temp.data % 2 == 0){
            if (evenhead == NULL){
                evenhead = temp
            }
        else
            temp2 = evenhead
            while (temp2.next ≠ NULL){
                temp2 = temp2.next
            }
            temp2.next = temp.
            temp.next = NULL
        }
    else{
        if (oddhead == NULL){
            odd head = temp.
        else(temp2 = odd head)
            while (temp2 ≠ NULL)
            temp2 = temp2.next.
            temp2.next = temp
            temp.next = NULL
        temp = temp.next
    end
}
```

Diagram:-

1 → 2 → 3 → 4 → 5 → 6 → NULL

2 → 4 → 6 → NULL even list

1 → 3 → 5 → NULL odd list.

7. Task 7:

Algorithm:-

→ use slow pointer to find midpoint.
- break the list from middle.
- reverse each half
- connect the tail of first half to the first node of second ~~head~~ half.

Pseudocode:-

```
void reserseBothHalve ( ){
    mid = slow
    first halfhead = head
    second halfhead = mid.next
    mid . next = NULL
  while (firsthalfhead ≠ NULL){
        reverse()   //calling reverse function for first half.
  while secondhalfhead != NULL
            revers
    temp2 = first halfhead.
    while temp2.next ≠ NULL
            temp2 = temp2.next

    temp2 . next = second halfhead.
  end
```

Diagram:-

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → NULL

1 → 2 → 3 → 4 → 5 → NULL

6 → 7 → 8 → NULL

5 → 4 → 3 → 2 → 1 → NULL 8 → 7 → 6 → NULL

5 → 4 → 3 → 2 → 1 → 8 → 7 → 6 → NULL.

8. Task 8:-

Algorithm:-

→ saving the value/design for each node.
→ If value matches the value for any other node in the list the node is deleted.
→ Starting temp from head. and moving it forward in each iteration.

Pseudocode:-

```
method    removeDuplicateDesign( ) {
        temp = head.
        while (temp.next ≠ NULL){
            temp = temp.next   // Last node.
            }
        ~~while temp~~
        temp2 = head
        while (temp2 != NULL)
            start = head
            while (start != NULL)
                if (start.data = temp2.data)
                    if (start == head)
                        removeFirst() // calling function for removing
                                         first node.
                    if (start == temp)
                        remove Last()
                    else
                        removeAfterSpecificValue(start.data)
                        start = start.next
                    ~~end if~~ }
                temp2 = temp2.next.
            }
        }
```

Diagram:-

1 → 2 → 3 → 1 → 4 → NULL
↑
temp2.data

1 → 2 → 3 → 1 → 4 → NULL
↑
start
temp2.data == start.data.

1 → 2 → 3 → 4 → NULL.

9. Task 9:

Algorithm:

- → Comparing data for each node.
- → comparing with the specific identifier. number.
- → if there is a match, we remove all the matching node.

Pseudocode:

```
method deleteValue (key)
  temp = head
  if (head.data == key)
    temp2 = head
    head = head.next
    delete (temp2)
  while (temp.next != NULL)
    if (temp.next == key)
      temp2 = temp.next
      temp.next = temp.next.next.
      delete (temp2)
    else : temp = temp.next.
  }

}
```

Diagram:

1 → 3 → 5 → 7 → 3 → 2 → 3 → NULL.

1 → 5 → 7 → 2 → NULL.

10. Task 10

Algorithm:-

we traverse the list in pairs while swapping every 2 nodes updating links and moving on to the next pair till null value is reached!

Pseudocode:-

```
void swappair ()
    temp = head
    while (temp != NULL and temp.next != NULL)
        temp2 = temp.data.
        temp.data = temp.next.data
        temp.next.data = temp2.

        temp = temp.next.next.
    }
}
```

Diagram:-

1 → 2 → 3 → 4 → 5 → 6 → NULL

next.data prev.data temp=temp.next.next

2 → 1 → 4 → 3 → 6 → 5 → NULL.

Doubly Linked List Tasks.

11. Task 1:-

Algorithm:-

we take a current node which starts from ahead.

then we update the prev and next pointer of current node.

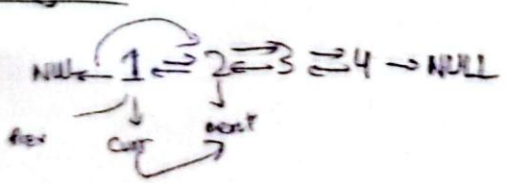
we update the position of current node.

Pseudocode:-

```
void reverseList ()
    curr = head
    while (curr != NULL)
        temp = curr.prev
        curr.prev = curr.next
        curr.next = temp

        curr = curr.prev   //because pointers are swapped.
        }
    }
```

Diagram:-



NULL ← 4 ⇄ 3 ⇄ 2 ⇄ 1 → NULL

12. Task 2

Algorithm

we take the song ~~into the~~ be navigated as a key.
→ we search for the song in the playlist ie doubly linked list
→ return the song if it is found.

Pseudocode:

```
void navigateSong(key)
    temp = head.
    while(temp.next != NULL){
        if (temp.data == key) {
            break
            } temp=temp.next.
    }
    if (temp != NULL) {
        if (temp.next ≠ NULL){
        print next song.}
        if (temp.prev != NULL){
        print << prevsong
}
```

Diagram: key is song 4.

NULL ← S1 ⇄ S2 ⇄ S3 ⇄ S4 → NULL nonext song

temp prevsong temp.data == key.

13. Task 3

Algorithm:

we create the following classes to navigate stores, sections and items:

- class Items
- class Stores
- class Sections
- class Inventory.

Pseudocode:

```
class Items{
        name
        next
    }

class Section{
        name
        itemhead
        next.
    }
class Store{
        name
        sectionhead
        next
    }
class Inventory{
        Storehead.

a) void addSection (name, storeName)
        Section S1 = new Section(name).
        Store temp = storehead;
        while (temp ≠ NULL){
            if (storename == tempname){
                break
            }
            temp = temp.next
        }
        if (store.sectionhead == NULL)
            store.sectionhead = S.
        store temp2 = store.sectionhead
        while (temp2 ≠ NULL)
            temp2 = temp2.next
        }
        temp2.next = S

    }
```

b)
```
void storeItem (itemname, section name, store name){
    Store temp1 = storehead.
    while(temp1 ≠ NULL)
        if (Storename == temp1 .name)
            break
        temp1 = temp1 .next
    }
    Section temp2 = temp1 .sectionhead.
    while(temp2 ≠ NULL){
        if (section name == temp2.nam){
            break }
        temp2 = temp2.next
        }
    Item i = new Item(name)
        if (temp2. itemhead == NULL) {
            temp2 .itemhead = i.
        }
    Item temp3 = temp2 .Itemhead.
        while (temp3 .next != NULL) {
            temp 3 = temp3.next
        }
    temp3.next = i
}
```

c)
```
void removeItem (name, storename, section name)
    // navigating store and section in a similar manner as the above method.
    Item temp = foundSection. itemhead.
        while(temp != NULL){
            if (name == temp .name)
                break
            temp=temp.next
        }
    temp.prev.next = temp .next   // removing item node.
    temp .next.prev = temp .prev.
}
```

d) void displaySectionItems (store name, section name)

```
    // navigating store and section as done previously.
    Item temp = section.itemhead.
    while (temp != NULL)
        print (temp.name).
        temp=temp.next.
    }
```

e) void displayStoreItems (store name)

```
    section temp = store.sectionhead.
    while (temp != NULL)
        Item temp2 = temp.itemhead.
        while (temp2 != NULL)
            print temp2.name.
            temp2=temp2.next.
        }
        temp =temp.next.
    }
```

<u>Diagram</u>

Inventory (Stores) S1 → S2 → S3

↓

Sections sec1 → sec2 → sec3.

↓

Items i1 → i2 → i3

14. Task 4:-

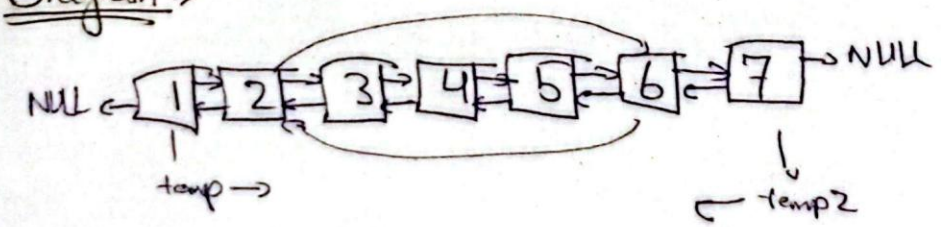
Algorithm:-

- we simultaneously traverse a loop from left and right ~~and righ~~ till they meet in the middle.
We control a counter variable which is incremented after each iteration.
We swap the even places using the counter.

Pseudocode:-

```
void SpecialSwap () {
    Node temp = head
    Node temp2 = head.
    while (temp2 .next != NULL){
        temp2 = temp2.next
    }
    int count = 1
    while (temp != temp2){
        if (count % 2 == 0 ){
            int temp data = temp.data
            temp.data = temp2 .data.
            temp2 .data = temp data.
            }
        temp = temp.next
        temp2 = temp2.prev.
        count ++
    }
}
```

Diagram:-



temp = temp2

even places are swapped!

Circular Linked Tasks:

15. Task 1 :.

Algorithm :-

⇒ We take two nodes :-

temp → Stores current value.

temp2 = stores previous node of temp.

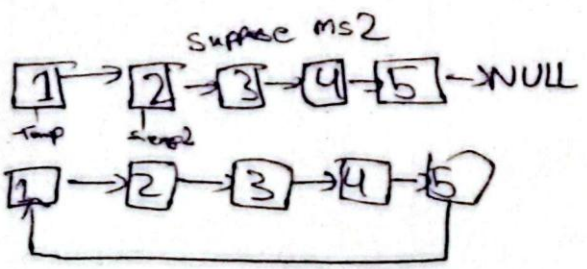
⇒ In each iteration we move M-1 nodes forward and delete the node.

⇒ Updating temp and temp 2 at the end of each iteration

Pseudocoder

```
void JosephusProblem (N, M) {
    Node temp = head
    Node temp2 = head
    while (temp2 -next != NULL);
        temp2 = temp2.next
    }
    while (temp.next != temp){
        for (int i = 0; i < M ; i++){
            temp 2 = temp
            temp = temp.next
        }
    temp2.next = temp.next
    delete (temp)
        temp = temp2.next
    }
    print "you will survive in place" << temp.data
    head = temp.
}
```

Diagram :-



The loop iterates and removes every M-1 value.

16. Task 2

Algorithm:-

→ to add a new task we create a new node for the task, make the last node as its previous and connect it with the head to ensure circulality.

→ To remove a tas we find the node update its neighbouring nodes and delete it.

→ to find next task we iterate though the list and return a pending taske

→ to display all tasks we iterater through the loop and output data of each task.

→ for updating tasks we find the node and update it.

Pseudocoder

a)
```
void addTask (task t) {
        Node nn = new Node()
        nn.data = t.
        if (head == NULL)
            head = nn
            head.next = head.
        else {
            Node temp = hed
            while (temp.next != head){
                temp = temp.next
            }
            temp.next = nn
            nn.next = head.
        }
```

b)
```
void removeTask (name)
    Node temp = head
    while (temp.next != head )
        if (temp.next.data.name == name){
            break.
        } temp = temp.next
    }
    temp.next = temp.next.next
    delete temp;
```

```
if (temp == head)
    temp 2 = head
    head = head.next
    delete (temp 2)
    lastNode.next = head.
    }
}

void getNextTasks ()
    Node temp = head
    while (temp.next != head) {
        if (temp.data.status == Pending) {
            return temp
            }
        }
        print "No Pending tasks".
        return.
    }

void displayAllTasks ()
    Node temp = head
    while (temp->next != head) {
        print (temp.data.name)
        print (temp.data.priority)
        print (temp.data.status)
        temp = temp.next
        }
    }

void update status () {
    // we find the node based on nname like we
did above.
temp.data.status = "completed" / "Inprogress" / "Pending".
}
```